CAB432 CLOUD COMPUTING | QUT | 2026

Assignment 3: Cloud Project

Project guidance and requirements checklist

Weighting	40% of final grade (graded out of 40)
Type	Individual
Due	**Fri 30 Oct 2026, 23:59** (all three items: report, video, YAML + source). Eligible for the 48-hour late period and extensions
Deliverables	(1) Report PDF to Canvas (2) 4-5 min demo video to Canvas (3) Infrastructure YAML + source code to Gradescope
AWS scope	CAB432 shared account, **ap-southeast-2**, tag **purpose = assessment 3**, resource names include student ID
Report limit	30 pages max incl. diagrams; excludes references and appendices
Do not	Continue working on Strobe (finished with A2)

About this document
This guide restructures the official assignment specification into checklists and a working plan. The Canvas specification and the CRA rubric remain the authority. Items marked *(suggested)* are planning suggestions, not official requirements.

Contents

1. What the assignment is really about 3

The four-step path . . . 3

How marks are awarded (CRA) . . . 3

Unit Learning Outcomes assessed . . . 3

2. Project timeline 4

Week-by-week plan . . . 5

3. Cloud patterns checklist 6

Additional patterns (any of, to reach eight or more) . . . 6

4. AWS rules and resource reuse 6

FAQ: working around account limits . . . 7

5. Learning on your own (ULO4) 7

6. Front end 7

7. The report (primary deliverable) 8

Report structure and suggested page budget . . . 8

Application implementation: questions to answer . . . 8

Cost and sustainability: method . . . 9

Reflection: prompts . . . 9

8. Demonstration video 9

9. Gradescope: infrastructure YAML and source code 9

10. GenAI rules and integrity 10

11. Example project ideas from the spec 10

12. Contacts and extensions 10

13. Final pre-submission checklist 11

Design and build . . . 11

Report . . . 11

Submission . . . 11

1. What the assignment is really about

Build a cloud application that uses multiple cloud compute services in a cost-effective, scalable architecture, then write a professional report about it. There is no fixed problem statement or scaffolding: the problem, scope and services are your choice, as long as the problem gives a **genuine reason to use several cloud services together and to demonstrate scaling**. You are also responsible for keeping the scope bounded and achievable.

The key question to answer throughout
Why does this need to run in the cloud, and why have you designed it this way?
The marks reward your understanding of how and why cloud services work together. The application is the context for demonstrating that understanding, not the primary focus.

The four-step path

Step	What to do
1. Pick a problem	Suitable complexity; genuinely benefits from several cloud services working together; can be stressed to demonstrate scaling. Ideally resume-worthy.
2. Design	Scalable, cost-effective architecture. Justify every key decision against scalability, security, reliability, maintainability and cost.
3. Implement & deploy	Build on AWS and **demonstrate that it scales**.
4. Report	Professional report explaining and justifying everything, with architecture diagrams, costing and sustainability discussion.

How marks are awarded (CRA)

- Analysing the problem and identifying suitable cloud solutions
- Designing and justifying an appropriate cloud architecture
- Implementing and evaluating the system, including its scalability and operation in the cloud
- Communicating through a professional report that uses evidence, reflection and technical reasoning

A working app matters, but it is not simply a functionality demo. Strong responses show *why* particular architectural approaches, workflows and services were appropriate. Check the full rubric under the Assessments tab on Canvas.

Unit Learning Outcomes assessed

ULO	Summary
ULO1	Discuss the elastic nature of cloud technologies and business models in technical, commercial and sustainability contexts
ULO2	Analyse requirements to select suitable cloud infrastructure and XaaS services
ULO3	Design and implement scalable cloud apps with industry-standard languages and APIs on public cloud, using a range of services
ULO4	Investigate cloud services on your own from vendor docs, code samples and other sources
ULO5	Communicate in written, verbal and visual forms about cloud apps and architectures

2. Project timeline

All three items are due **Fri 30 Oct 2026 at 23:59** (Canvas). The plan targets **submission-ready by Wed 28 Oct**, leaving Thu 29 - Fri 30 Oct as buffer for proofreading, upload problems or last fixes. *(Milestone dates are suggestions.)*

A2 presentation date (if still pending): _____

Workstream	Wk 10 29 Sep-4 Oct	Wk 11 5-11 Oct	Wk 12 12-18 Oct	Wk 13 19-25 Oct	Final push 26-28 Oct	Buffer 29-30 Oct
Scope and requirements	■					
Architecture and diagrams	■	■		▪		
Core build (storage, compute, API)		■				
Scaling, caching, async			■			
Front end		▪	■			
Self-learning log (ULO4)	▪	▪	▪	▪		
Load testing and evidence				■		
Cost estimate (Pricing Calculator)				■		
Demo video				▪	■	
Report writing		▪	▪	■	■	
YAML, final checks, submit					■	▪
Milestone	◆ Scope locked	◆ End-to-end working	◆ Feature freeze	◆ Evidence + draft	◆ Wed 28: ready	◆ Submit

■ Main effort ▪ Ongoing / lighter effort

Week-by-week plan

Week	Tasks	Milestone / done when
Wk 10 29 Sep-4 Oct	Brainstorm and pick a problem that genuinely needs several services and can be stressed. Write functional and non-functional requirements (user stories). Pick 8+ patterns (section 3), including non-Lambda auto-scaling and caching. Sketch the vendor-agnostic diagram. Choose the ULO4 self-learning topic and start the learning log. Run AWS Resource Explorer; list resources to reuse, retag or delete.	◆ **Sun 4 Oct: scope locked.** One-paragraph problem statement, requirements list, pattern list, draft agnostic diagram.
Wk 11 5-11 Oct	Map each agnostic component to an AWS service; draft the AWS diagram. Build the core: storage, compute, API (under an `/a3` prefix if reusing API Gateway), Cognito. Tag everything *purpose = assessment 3*, names with student ID. Start a minimal front end. Begin drafting Introduction and requirements.	◆ **Sun 11 Oct: end-to-end happy path works** through the front end.
Wk 12 12-18 Oct	Implement auto-scaling (metric- or request-based), load distribution (dispatcher-push or worker-pull) and the caching layer. Add async workflow (queue / pub-sub), error handling, retries and timeouts. Set up monitoring (logs, metrics, dashboard). Finish the front end. Keep notes on decisions and trade-offs as you go.	◆ **Sun 18 Oct: feature freeze.** Scaling, caching and async all work.
Wk 13 19-25 Oct	Load test to show scaling out and back in; capture CloudWatch graphs and screenshots (12 pt legible). Run the ~15-minute usage experiment; build the Pricing Calculator estimate for 50 concurrent users and save the share link. Record video clips (scaling, caching, async). Finalise both diagrams. Draft the vendor-agnostic and AWS sections.	◆ **Sun 25 Oct: all evidence captured;** Intro + both architecture sections drafted.
Final push Mon 26-Wed 28 Oct	Draft Application implementation (data, schemas, flows, sequence diagrams, error handling), Cost and sustainability, and Reflection. Write the Executive summary last. Add references, appendices and title page with AI declaration. Edit the video to 4-5 min. Fill in the infrastructure YAML. Run the final checklist (section 13) and check the page count.	◆ **Wed 28 Oct: submission-ready.** Report PDF, video, YAML and source all final.
Buffer Thu 29-Fri 30 Oct	Proofread with fresh eyes. Submit all three items on Thursday if possible: report and video to Canvas (click **Submit** on the video), YAML and source to Gradescope, then check the Gradescope result. Keep resources running and tagged until marking is safe.	◆ **Submitted** before Fri 30 Oct, 23:59.

If you fall behind

If a week slips, protect the order: **scaling and caching working** first (they're the must-haves), then **evidence** (graphs, video clips, cost link), then report polish. A simpler app with clearly demonstrated scaling beats a richer app with none. The 48-hour late period (to about Sun 1 Nov, 23:59) and formal extensions exist, but don't plan around them.

3. Cloud patterns checklist

Consider, use and discuss the design patterns covered in CAB432. **Choose at least eight**, covering every required category below. Emphasise vendor-agnostic concepts over AWS specifics.

Must-haves
Auto-scaling and caching are particularly important because you have not demonstrated them before. **Lambda's built-in scaling does not count** as auto-scaling, and Lambda does not count for load distribution either.

Category (pick at least one)	Options	Planned choice
Data storage & persistence	☐ Structured data store ☐ Unstructured data store	
Caching	☐ Edge caching ☐ In-memory caching	
Service-to-service communication	☐ API gateway ☐ Message queues ☐ Publish-subscribe messaging ☐ Polling ☐ Request callback	
Compute	☐ Serverless computing ☐ Virtual machines ☐ Containers	
Scaling (not Lambda)	☐ Request-based scaling (e.g. start a new container/VM per user request) ☐ Metric-based scaling (e.g. CPU, or queued tasks per worker)	
Load distribution (not Lambda)	☐ Dispatcher-push (load balancer forwards to workers) ☐ Worker-pull (workers take tasks from a queue)	

Additional patterns (any of, to reach eight or more)

☐ Microservices	☐ Event-driven architectures
☐ Orchestration	☐ Identity, security and RBAC
☐ Configuration management	☐ Interfaces (CLI, WebUI, API, WebSocket)
☐ Networking and DNS	☐ Monitoring (logs, metrics, costing)
☐ Managed vs unmanaged services	☐ AI/ML services and systems
☐ Garbage collection	☐ Public vs private access (app layer)
☐ Stateless and/or stateful design	☐ Fault tolerance and resilience
☐ Proxies and reverse proxies	☐ Resilience patterns (retries, timeouts, circuit breakers)

Tally: required categories (6) + additional = _____ patterns (target: 8 or more, each clearly discussed in the report)

4. AWS rules and resource reuse

- Stay in the CAB432 shared account, region **ap-southeast-2**.
- Tag everything **purpose = assessment 3**. Use **AWS Resource Explorer** to find existing resources and retag any older ones you reuse so they aren't deleted.

- If A2 is not yet presented, keep A2 resources tagged *assessment 2* and relabel after the presentation.
- Resource names should include your **student ID**.
- Keep it low-cost: no large always-on resources. **Scale-to-zero and pay-per-use** are strongly encouraged.
- Use services already granted, drawing on covered services **plus features you investigate yourself**.

FAQ: working around account limits

Situation	Answer
Only one API Gateway	Add a path prefix for new endpoints, e.g. `/a3/foo`, `/a3/bar`
Only one Cognito user pool	Keep using it. No relabel needed.
Running out of DynamoDB tables	Add a sort key. Objects in one table don't need identical fields.
Reuse ECS cluster?	Yes. No relabel needed.
Delete old resources?	Yes, please, unless still needed for the A2 presentation.

5. Learning on your own (ULO4)

Some services, features and patterns are introduced in lectures but deliberately **not covered in practicals**. Investigating them yourself is assessed, and the learning must be **visible in the report**. Examples: applying the AWS Well-Architected Framework, or adopting managed observability or discovery tools. Pick something relevant to your solution and show you learned it from vendor documentation and other sources.

☐ Topic(s) chosen: _____

☐ Keep a running log of docs read, decisions changed and problems hit (feeds the Reflection section)

6. Front end

Any client-facing interface works: web app (e.g. React), desktop app, or a developer-facing one such as a VS Code extension, a Postman collection, or a TUI (e.g. Textual). It only needs enough functionality to interact with and **demonstrate how the architecture's components work together**. Front-end complexity is not assessed.

> **GenAI and the front end**
> GenAI may help write the front end (boilerplate, UI components, simple functionality), but you must understand the code and be able to explain how it interacts with the cloud architecture.

7. The report (primary deliverable)

A highly professional engineering report for a cloud engineering team or technical stakeholder. Describe **your actual system** with specific architectural descriptors, project-specific evidence and honest reasoning: what you built, why, trade-offs, and limitations. Avoid generic prose that could describe any cloud app.

- **Length:** 30 pages max including diagrams; references and appendices excluded.
- **Three core sections:** vendor-agnostic architecture (*what and why*), vendor-specific implementation (*how in AWS*), application implementation (*how the app uses it*).
- **Diagrams:** created digitally, accurate, directly represent your system. Text must be legible at ~12 pt on an A4 page. If too complex, **split into multiple focused diagrams** rather than shrinking. Tools: CloudCraft, draw.io (AWS shape sets), others.
- **Screenshots:** text must appear at 12 pt; landscape pages allowed; every one captioned and referenced in the text.

Report structure and suggested page budget

Section	Must include	Pages (sugg.)
Title page	Title: **CAB432 Assignment 3 2026**; subtitle: app name; name and student ID; declaration of AI use and other acknowledgements	1
Executive summary	**One page.** App, purpose, key architecture aspects; what the report covers; major decisions and patterns demonstrated	1
Table of contents	Self-explanatory	1
Introduction	Problem statement: what you built and why. Functional and non-functional requirements (user stories or similar). A few captioned screenshots of core features	3
Vendor-agnostic architecture	High-level diagram with no vendor products: compute, storage, communication and networking, IAM, other infrastructure. Design and selected patterns; reflection on scaling, async workflows, observability; limitations and fixes. Key decisions, integration, trade-offs; scalability, security, reliability, maintainability	6
Vendor-specific implementation	Second, more detailed diagram with AWS icons identifying compute, storage, networking, communication, identity, etc. Why these AWS services, how they integrate, trade-offs; scalability, security, reliability, maintainability	6
Application implementation	Application logic, data flows and service interactions (see questions below). **No code pasted.** Use sequence diagrams, flow diagrams, algorithms	5
Cost and sustainability	AWS Pricing Calculator estimate at 50 concurrent users; per-service totals; public share link; how scaling affects cost; sustainability implications	3
Reflection	Learning on your own (see prompts below)	2
References & appendices	Docs and sources cited; supporting material only (extra diagrams, config, detailed test results). Not counted in the 30 pages	n/a

Suggested total: about 28 pages, leaving buffer under the 30-page limit. It's safest to assume the title page, summary and contents count toward the limit.

> Don't use appendices to avoid discussing important information. Key decisions, evidence and explanations belong in the main report.

Application implementation: questions to answer

☐ What data do users and the app generate, process and store? Which services store it, and why?

- ☐ What data is exchanged between components/microservices? Communication mechanisms, message formats, data schemas, with example data objects.
- ☐ How does data flow from the initial user request to the final result?
- ☐ How does the app detect, handle and recover from infrastructure errors: service failures, invalid input, network issues, unsuccessful requests?
- ☐ How does it behave when demand increases or a component becomes unavailable?

Cost and sustainability: method

- ☐ Estimate monthly cost at an average of **50 concurrent users** (or a smaller stated number if the app can't support 50; say so in the report).
- ☐ Run the app yourself for ~15 minutes and read CloudWatch / monitoring metrics (CPU, RAM, bucket storage, etc.) to extrapolate. Order of magnitude is what matters.
- ☐ Build the estimate in the **AWS Pricing Calculator** (calculator.aws). Click **Share** and put the **public link** in the report.
- ☐ Summarise the total price **per AWS service**. Ask the teaching team before using any other tool.
- ☐ Discuss how the design stays cost-effective, how scaling changes cost, and sustainability implications.

Reflection: prompts

- Which unfamiliar services, technologies or patterns did you research, and why were they relevant?
- How did you use reference documentation to evaluate or improve the design?
- Specific examples where research led you to **adopt, modify or reject** an approach.
- Difficulties learning or implementing unfamiliar tech, and what you learned from them.
- What you'd investigate further or do differently next time.

Focus on what you learned, how, and how it shaped decisions, not a list of resources consulted.

8. Demonstration video

- ☐ Length **4 to 5 minutes**, submitted to **Canvas**. Click **Submit** after uploading.
- ☐ Shows the system working, including evidence of **scaling**, **caching** and **asynchronous behaviour**.
- ☐ "Look, it works" evidence, not explanation (that belongs in the report). Only say what you're showing.
- ☐ Screen capture, not a phone recording. Text must be readable; zoom in (Ctrl +).
- ☐ Clips can be recorded separately and edited together; long waits can be cut down.
- ☐ Tools: OBS Studio, or Canvas's built-in *Record media* option.

9. Gradescope: infrastructure YAML and source code

Same process as Assignment 2. Submit the YAML (template: `submission-a3-infra.yml`) **and all application source code**. Gradescope only checks that declared resources exist, are tagged correctly and are in-region. It doesn't test end-to-end.

- List **every** resource your solution depends on.
- `type`: kind of resource (e.g. Lambda function, EC2 Auto Scaling group, Application Load Balancer, CloudWatch dashboard).
- `arn`: the resource's unique identifier (ARN or other ID/endpoint).
- `description`: one plain-language sentence on its purpose in your solution.

```
cloudInfrastructure:
  - type: EC2 Auto Scaling group
    arn: <arn-or-other-identifier>
    description: <one-line description>

  - type: Application Load Balancer
    arn: <arn-or-endpoint>
    description: <one-line description>

  - type: Lambda function
    arn: <arn>
    description: <one-line description>

metaData:
  qutUsernameTag: <qut-username@qut.edu.au>
```

10. GenAI rules and integrity

Deliverable	GenAI allowed?
Report	**Must be drafted entirely by you.** AI may only refine your writing style. Declare AI use on the title page.
Front end	Yes: boilerplate, UI components, simple functionality. You must understand and be able to explain it.
Interview	The unit coordinator may ask you to explain your design and implementation in an interview.

11. Example project ideas from the spec

Optional starting points. You're encouraged to choose your own.

Idea	Concept	Patterns it naturally exercises
Py tutor	Online Python editor + console + LLM tutor chat. No login; shareable session links. User code runs in sandboxed containers.	Request-based scaling (container per session), security/sandboxing, AI/ML, WebSocket, garbage collection of idle containers
Wallpaper of the day	Resize images from approved sources (e.g. NASA) to any screen size; random pick option.	Long-term unstructured store, in-memory cache, edge cache (three cache layers)
Web transcoder	Upload a video, download it as H.264 via ffmpeg. Files deleted after 2 hours.	Message queue, worker-pull, metric-based scaling on queue depth, garbage collection

12. Contacts and extensions

- One due date covers all three items; submit on time or apply for a formal extension.
- AEP with oral presentation adjustments: email **cab432@qut.edu.au**. AEP for extension only: use the formal extension process.
- Other special circumstances or concerns: email **cab432@qut.edu.au** with **"A3"** in the subject.
- Marks are released only after final grades (QUT MOPP). Reviews can be requested after that.

13. Final pre-submission checklist

Design and build

- ☐ Problem genuinely needs several cloud services and can be stressed to show scaling
- ☐ 8+ patterns, covering storage, caching, communication, compute, scaling and load distribution
- ☐ Auto-scaling implemented without relying on Lambda; scaling demonstrated under load
- ☐ Caching implemented and demonstrated
- ☐ At least one self-learned service/feature/pattern (ULO4)
- ☐ Scale-to-zero / pay-per-use where possible; no large always-on resources
- ☐ All resources in ap-southeast-2, tagged purpose = assessment 3, names include student ID
- ☐ Reused older resources retagged (after A2 presentation if applicable)

Report

- ☐ Title page: "CAB432 Assignment 3 2026", app name, name and student ID, AI declaration
- ☐ Executive summary fits on one page
- ☐ Table of contents
- ☐ Introduction with functional and non-functional requirements and captioned screenshots
- ☐ Vendor-agnostic diagram (no vendor names/icons) covering compute, storage, networking, IAM
- ☐ AWS-specific diagram with AWS icons, more detailed
- ☐ Scaling, async workflows, observability and limitations discussed
- ☐ Trade-offs plus scalability, security, reliability, maintainability discussed at both levels
- ☐ Application implementation: data, schemas and example objects, flows, error handling, behaviour under load; no code
- ☐ Pricing Calculator per-service totals + public share link; 50 users (or stated number)
- ☐ Sustainability discussion
- ☐ Reflection on learning on your own
- ☐ References; appendices used only for supporting material
- ☐ All diagram/screenshot text legible at ~12 pt; every figure captioned and referenced
- ☐ 30 pages or fewer (excluding references and appendices)
- ☐ Report drafted by you; AI used only for style refinement

Submission

- ☐ Report PDF uploaded to Canvas
- ☐ Video (4-5 min, screen capture) uploaded to Canvas and **Submit** clicked
- ☐ Infrastructure YAML listing every dependent resource, with a one-sentence description each, on Gradescope
- ☐ All application source code on Gradescope
- ☐ QUT username in `metaData.qutUsernameTag`